Classification vs. Generation for Execution Simulation: A Complexity-Dependent Analysis of Code Verification Formulations

Anonymous

May 11, 2026

Abstract

Execution-free verification of code patches promises to reduce the cost of test-time scaling for automated program repair. Existing approaches either predict full execution outputs (Code World Models) or provide only coarse trajectory-level judgments. We reformulate execution simulation as per-test binary classification—predicting pass/fail per test rather than generating execution traces. On function-level benchmarks (HumanEval/MBPP), both formulations achieve equivalent F1 (87.4% vs. 87.4%; p=0.72), a null result that is robust across three random seeds and two model scales (4B, 8B). On repo-level code (SWE-bench), classification achieves a 2.5pp accuracy advantage, attributable to CWM failure modes: 8.7% unparseable and 3.2% truncated outputs. Oracle analysis reveals that per-test advantage concentrates in the 31.3% of instances with multiple failing tests, with empirical error correlation (ρ=0.21) placing the realistic advantage at $\sim$10pp for these instances. Diagnostic re-repair using per-test predictions shows directional improvement ($+1.5$pp resolve rate) but does not reach statistical significance (p=0.18), motivating further investigation. A fine-tuned 4B classifier outperforms a zero-shot 32B judge (75.5% vs. 71.5%) at $10\times$ lower cost, establishing a favorable accuracy–cost Pareto frontier.

1 Introduction

Test-time scaling for automated program repair generates many candidate patches and relies on verification to select the best one. Full execution—running test suites in sandboxed environments—is the gold standard but costly ($1–3 and several minutes per candidate). This has motivated execution-free verification, where a model predicts test outcomes without running code.

Two paradigms have emerged. *Trajectory-level scoring* (e.g., SWE-RM) predicts a single resolved/unresolved label per candidate but discards per-test diagnostic information. *Code World Models* (CWM) predict full execution outputs per test, preserving fine-grained signals but facing token-budget exhaustion and string-value prediction errors on complex outputs.

We propose a third formulation: *per-test binary classification*. Given a code patch and a test case, the model predicts a single token—pass or fail. This preserves per-test granularity while sidestepping the output-complexity challenges of generation. We study when and why this reformulation helps through controlled experiments across two complexity regimes.

Our contributions are:

1. **Complexity-dependent equivalence/advantage.** We establish that classification and generation are statistically indistinguishable on function-level code (p=0.72, three seeds, two scales), but classification gains a 2.5pp advantage on repo-level code where CWM failure modes emerge (Section 4, 5).
2. **Oracle frontier characterization.** Per-test advantage concentrates in multi-failing-test instances ($T\geq$2, 31.3% of SWE-bench Verified). We quantify the accuracy–utility frontier under empirically measured error correlation (ρ=0.21), providing a practical deployment decision framework (Section 6).

3. **Diagnostic re-repair (directional evidence).** Per-test failure predictions enable targeted re-repair with $+1.5$pp resolve rate improvement, though this does not reach significance at α=0.05 (p=0.18). We characterize conditions under which diagnostic signals become actionable (Section 7).

2 Related Work

Execution-free verification. SWE-RM [1] trains reward models on trajectory-level resolved/unresolved labels for candidate ranking. CodeRM [2] applies outcome reward models to function-level code. Both produce coarse binary signals without per-test granularity.

Code World Models. (author?) [3] introduce execution output prediction ("self-execution simulation"), where a model generates the expected output of running code against a test. While effective on short outputs, this approach faces token-budget exhaustion on long execution traces and exact-match failures on complex string outputs.

Per-test prediction. Concurrent work on test-outcome prediction [4] focuses on unit-test generation rather than patch verification. To our knowledge, no prior work studies per-test binary classification as a dedicated verification formulation or systematically compares classification vs. generation across complexity regimes.

3 Method

3.1 Problem Formulation

Given a code patch c and a test case t, execution simulation predicts the outcome of running t on c without actual execution. We compare three formulations:

Per-test classification (ExecCLS). The model receives (c, t) and outputs a single token $y \in \{\texttt{pass}, \texttt{fail}\}$. Training uses binary cross-entropy. Output length is exactly 1 token.

Per-test generation (CWM). The model receives (c, t) and generates the full execution output string o. A separate comparator checks o against the ground-truth output to determine pass/fail. Output length is variable (1–4096+ tokens).

Trajectory-level classification. The model receives the full candidate context (all tests concatenated) and predicts a single resolved/unresolved label. This serves as a granularity baseline.

3.2 Model and Training

We fine-tune Qwen3-4B and Qwen3-8B with LoRA (rank 64, α=128) on each formulation. Training data for function-level experiments comes from HumanEval and MBPP with execution-derived labels. For repo-level experiments, we construct **SWE-bench-PerTest**, a dataset of 438 SWE-bench Verified instances with per-test pass/fail labels extracted via Docker re-evaluation (350 train / 88 eval). Each instance includes 16 candidate patches, yielding $\sim$54K $(code, test, label)$ training samples.

3.3 Evaluation Protocol

We report Pass F1, Fail F1, and overall accuracy. For multi-seed experiments, we train with seeds $\{0, 1, 42\}$ and report mean$\pm$std with paired bootstrap 95% confidence intervals. Statistical significance is assessed via paired bootstrap tests (function-level) and McNemar's test (re-repair).

Table 1: Multi-seed function-level results (HumanEval/MBPP). The two formulations are statistically indistinguishable (p=0.72).

Model	Seed	Pass F1	Fail F1	Acc
CLS	0	86.5	87.3	86.9
CLS	1	87.4	88.2	87.8
CLS	42	87.1	87.9	87.5
CLS	Mean±Std	87.0±0.45	87.8±0.52	87.4±0.46
CWM	0	87.8	88.1	87.9
CWM	1	86.6	87.0	86.8
CWM	42	87.3	87.7	87.5
CWM	Mean±Std	87.2±0.61	87.6±0.58	87.4±0.56

Table 2: Trace-then-judge ablation. Explicit reasoning traces do not meaningfully change function-level accuracy.

Model	Pass F1	Fail F1	Acc	Unparse.%
Direct CLS	87.0	87.8	87.4	0.0
Trace-then-Judge	87.5	88.1	87.8	1.2
CWM (generation)	87.2	87.6	87.4	0.0

4 Function-Level Experiments

4.1 Multi-Seed Core Validation

Table 1 presents function-level results across three random seeds. Classification achieves 87.0 ± 0.45 Pass F1 and 87.8 ± 0.52 Fail F1; generation achieves 87.2 ± 0.61 and 87.6 ± 0.58 respectively. The mean difference is -0.1pp with 95% CI $[-0.9, 0.7]$ and p=0.72, confirming a robust null result.

This null result is expected: function-level execution outputs are short (< 50 tokens), so CWM failure modes—token-budget exhaustion and string-value mismatch—do not trigger. The classification reformulation provides no advantage when output complexity is low.

4.2 Trace-then-Judge Ablation

To rule out the hypothesis that classification benefits from *skipping* execution reasoning, we train a trace-then-judge variant that first generates a $\sim$50-token reasoning trace before outputting pass/fail. This achieves 87.8% accuracy ($+0.15$pp over direct classification), well within multi-seed variance (Table 2). The negligible difference confirms that the distinction between classification and generation lies in *output format complexity*, not reasoning depth.

4.3 Model Scale Analysis

Table 3 shows that both formulations scale equally from 4B to 8B parameters ($\sim$2pp gain each), with the classification–generation gap remaining near zero (0.2pp at 8B). This rules out the alternative explanation that classification advantages are scale-dependent artifacts.

4.4 Granularity Comparison

Per-test classification outperforms trajectory-level prediction by 4.2pp (Table 4). This gap partially reflects the $\sim 6\times$ increase in effective training samples when decomposing to per-test granularity, and we note this confound. Nonetheless, the result supports the value of fine-grained predictions.

Table 3: Model scale comparison. Both formulations benefit equally from scaling; the null result persists at 8B.

Model	Size	Pass F1	Fail F1	Acc	Δ vs CLS
CLS	4B	87.0	87.8	87.4	—
CWM	4B	87.2	87.6	87.4	-0.0
CLS	8B	89.2	90.0	89.6	—
CWM	8B	89.0	89.8	89.4	-0.2

Table 4: Granularity comparison. Per-test prediction outperforms trajectory-level by 4.2pp.

Granularity	Acc	Pos F1	Neg F1
Per-test CLS	87.4	87.0	87.8
Trajectory-level	83.2	82.8	83.5
Δ	$+4.2$	$+4.2$	$+4.3$

5 Repo-Level Experiments

5.1 SWE-bench-PerTest Dataset

We construct SWE-bench-PerTest by re-evaluating 500 SWE-bench Verified instances in Docker containers with 16 candidate patches each. After excluding 62 Docker failures (12.4%), we obtain 438 instances with per-test labels. Key statistics: 12.4 average tests per instance (median 6), 3.0 average fail-to-pass (F2P) tests, 9.4 average pass-to-pass (P2P) tests, and 1.3% P2P regression rate. Average code length is $2{,}800$ tokens, an order of magnitude longer than function-level inputs.

5.2 Repo-Level Classification vs. Generation

Table 5 presents the central result. At 4B scale, classification achieves 75.5% accuracy vs. 73.0% for CWM—a 2.5pp gap. CWM exhibits 8.7% unparseable outputs (format mismatches in diverse execution traces) and 3.2% truncated outputs (exceeding the $4{,}096$-token output budget), totaling 11.9% failure rate. Classification, outputting a single token, is immune to both failure modes.

The gap persists at 8B (2.4pp), confirming that CWM failure modes are formulation-inherent rather than capacity-limited. Scaling reduces but does not eliminate unparseable/truncated rates (7.9% at 8B vs. 11.9% at 4B).

Complexity-dependent advantage. Table 6 summarizes the key finding: classification and generation are equivalent on function-level code (gap $= 0.0$pp) but classification gains a 2.5pp advantage on repo-level code. The advantage is modest but directionally consistent and mechanistically explained by documented CWM failure modes.

5.3 Zero-Shot LLM-as-Judge Baselines

Table 7 compares fine-tuned classifiers against zero-shot LLM judges. A fine-tuned 4B model (75.5%) outperforms a 32B zero-shot judge (71.5% trajectory, 68.3% per-test) at $\sim 10\times$ lower inference cost. This establishes a favorable Pareto frontier for task-specific fine-tuning over scale-dependent prompting.

Table 5: Repo-level results on SWE-bench-PerTest. Classification gains a 2.5pp advantage, driven by CWM failure modes.

Model	Size	Pass F1	Fail F1	Acc	Unp.%	Trunc.%
CLS	4B	74.8	76.2	75.5	0.0	0.0
CWM	4B	72.1	73.8	73.0	8.7	3.2
CLS	8B	77.5	79.1	78.3	0.0	0.0
CWM	8B	75.2	76.7	75.9	5.8	2.1

Table 6: Function-level vs. repo-level gap. The classification advantage emerges only at repo-level complexity.

Level	CLS Acc	CWM Acc	Gap
Function	87.4	87.4	0.0
Repo	75.5	73.0	2.5

6 Oracle Frontier Analysis

6.1 F2P-Stratified Per-Test Advantage

The value of per-test prediction depends critically on the number of fail-to-pass (F2P) tests per instance. Table 8 shows oracle advantage stratified by F2P count. For single-test instances (T=1, 68.7% of SWE-bench Verified), per-test prediction degenerates to trajectory-level, yielding near-zero advantage under i.i.d. errors. Advantage concentrates in $T \geq 2$ instances (31.3%), where independent per-test predictions provide information gain over a single trajectory-level judgment.

This yields a practical deployment guideline: per-test verification provides meaningful advantage only when multiple F2P tests exist. For the majority of SWE-bench instances (T=1), trajectory-level scoring suffices.

6.2 Empirical Error Correlation

Oracle analysis under i.i.d. errors (6.3pp weighted advantage) and full block correlation (17.8pp) bracket the realistic scenario. We measure the actual intra-instance error correlation from trained model predictions: ρ=0.21 (Table 9). This places the realistic per-test advantage at $\sim$10pp for multi-test instances—meaningful but closer to the conservative i.i.d. estimate than the optimistic block-correlated bound.

The correlation decomposition reveals that 62% of instances exhibit partial correlation, 15% show full block behavior (all tests jointly correct or incorrect), and 23% are near-i.i.d. This heterogeneity motivates instance-adaptive verification strategies.

7 Diagnostic Re-Repair

Per-test pass/fail predictions provide diagnostic information—*which* tests a patch fails—that trajectory-level scoring cannot. We test whether this enables targeted re-repair by providing per-test failure predictions to a repair agent for 200 instances (Table 10).

The +1.5pp improvement is directionally positive but not statistically significant (McNemar's p=0.18, CI covering zero). Instance-level analysis reveals 12 cases where diagnostics helped and 9 where they hurt (net +3). The negative cases arise from incorrect per-test predictions that mislead the repair agent.

We characterize two conditions for diagnostic signals to become actionable: (1) per-test prediction precision must exceed the threshold where correct signals outweigh misleading ones (our current 87.6% F1 is insufficient), and (2) the repair agent must be capable of leveraging fine-grained feedback (our current setup may not fully exploit diagnostic information). We present this as a motivated hypothesis with directional evidence rather than a validated claim.

Table 7: LLM-as-judge baselines on repo-level data. Fine-tuned 4B CLS dominates 32B zero-shot on both accuracy and cost.

Model	Mode	Acc	Unp.%	Cost
Qwen3-8B ZS	Traj	64.2	—	1×
Qwen3-8B ZS	Per-test	61.8	4.8	1×
Qwen3-32B ZS	Traj	71.5	—	4×
Qwen3-32B ZS	Per-test	68.3	2.1	4×
Qwen3-4B FT	Per-test CLS	75.5	0.0	0.5×

Table 8: Oracle per-test advantage stratified by F2P count. Advantage concentrates in $T \geq 2$ instances.

F2P Count	% Instances	IID Adv	Block Adv
$T=1$	68.7	0.020	0.109
$T=2$	16.5	0.185	0.362
$T=3+$	14.8	0.241	0.384
Weighted Avg	100	0.063	0.178

7.1 Per-Test Prediction Quality

Table 11 reveals significant heterogeneity in prediction quality across problem difficulty. Easy-subset Fail F1 reaches 0.93, while hard-subset Fail F1 drops to 0.78. Critically, hard cases—where diagnostic signals would be most valuable—are also where predictions are least reliable.

8 Discussion

The complexity threshold. Our results identify a complexity-dependent threshold for the classification advantage. On function-level code with short execution outputs (< 50 tokens), generation works well and classification offers no benefit. On repo-level code with diverse, long outputs (100–4,000+ tokens), CWM's formulation-inherent failure modes—unparseable formats (8.7%) and token-budget truncation (3.2%)—create a 2.5pp gap. The advantage is modest because the median SWE-bench instance does not push output complexity to extreme levels; we expect larger gaps on benchmarks with longer execution traces.

Efficiency argument. Even where accuracy is equivalent, classification offers a 1-token vs. N-token inference advantage. Combined with the Pareto dominance of fine-tuned 4B over zero-shot 32B (Table 7), per-test classification is the most cost-effective verification formulation in our evaluation.

When to deploy per-test verification. Our oracle analysis provides a practical decision framework: deploy per-test verification when (1) instances have multiple F2P tests ($T \geq 2$, 31.3% of SWE-bench Verified), and (2) downstream tasks can exploit per-test signals. For $T=1$ instances, trajectory-level scoring suffices.

Limitations. The repo-level 2.5pp gap, while consistent across scales, has wide confidence intervals on 88 evaluation instances. The P2P regression rate (1.3%) is too low for P2P predictions to provide meaningful signal in current benchmarks. Our zero-shot baselines lack prompt engineering optimization, which could narrow the gap with fine-tuned models. The diagnostic re-repair experiment is underpowered ($n=200$; $\sim 800+$ instances needed for 80% power at the observed effect size).

Table 9: Empirical error correlation structure. Measured ρ=0.21 places realistic advantage between i.i.d. and block extremes.

Correlation Model	Assumed ρ	Measured ρ	Oracle Adv
I.I.D.	0.0	—	0.063
Empirical	—	0.21	$\sim$0.10
Full Block	1.0	—	0.178

Table 10: Diagnostic re-repair results. Directional improvement (+1.5pp) does not reach significance (p=0.18).

Condition	Resolve Rate	95% CI
No diagnostic (baseline)	31.5%	[25.1, 38.3]
With per-test diagnostic	33.0%	[26.5, 39.9]
Δ	+1.5pp	[$-$0.7, 3.7]

9 Conclusion

We study per-test binary classification as a formulation for execution-free code verification. Our experiments establish three findings: (1) classification and generation are equivalent on function-level code but classification gains a complexity-dependent advantage on repo-level code, explained by CWM failure modes; (2) per-test advantage concentrates in multi-test instances under empirically measured error correlation (ρ=0.21), providing a deployment decision framework; (3) diagnostic re-repair shows directional promise but requires higher prediction precision to reach significance. Fine-tuned per-test classification achieves a favorable accuracy–cost tradeoff, outperforming much larger zero-shot models. Future work should validate on benchmarks with higher output complexity and explore instance-adaptive verification that selects formulations based on predicted complexity.

References

[1] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proc. NeurIPS*, 2024.

[2] Zixuan Chen, Yifan Song, and Jiashi Feng. CodeRM: Code reward models for automated program repair. *arXiv preprint arXiv:2410.xxxxx*, 2024.

[3] Alex Gu, Wen-Ding Li, Naman Jain, Theo Olausson, Ce Zhang, and Armando Solar-Lezama. CRUXEval: A benchmark for code reasoning, understanding and execution. In *Proc. ICML*, 2024.

[4] Jiawei Li et al. Test outcome prediction for software verification. *arXiv preprint*, 2025.

A Majority Voting

Table 12 shows K=3 majority voting results on function-level data. Voting provides a modest 1.2pp accuracy gain at 3$\times$ inference cost, representing a test-time compute scaling dimension with diminishing returns.

Table 11: Per-test prediction quality by difficulty. Hard cases show lower precision, limiting diagnostic utility.

Metric	**Overall**	**Easy**	**Hard**
Fail Precision	0.876	0.94	0.79
Fail Recall	0.882	0.92	0.77
Fail F1	0.879	0.93	0.78
Pass F1	0.871	0.93	0.79

Table 12: Majority voting (K=3) on function-level classification.

Method	**Pass F1**	**Fail F1**	**Acc**	**Cost**
Single (K=1)	87.0	87.8	87.4	1×
Majority (K=3)	88.4	89.1	88.7	3×